

Relatório EP4 MAC0420

NUSP: 15485062

Julho 2026

1 O que precisa ser refatorado

Para este exercício, iremos refatorar o arquivo *sistema.py*. O código original é possui diversos problemas arquiteturais. Dentre esses problemas, temos:

- Nomes de variáveis e classes não são descritivos
- Classes pouco abstraídas
- Baixa coesão (god function)
- Separação entre regras de negócio, entrada e saída
- Pouca organização e legibilidade

Veja abaixo como é a implementação do único método presente no arquivo original:

```
def x(self):
    s = 0.0
    d = 0.0
    for i in self.c:
        v = i.b * i.c
        s = s + v
        if i.a == "LIVRO":
            d = d + v * 0.05
        elif i.a == "ELETRONICO":
            d = d + v * 0.02
        elif i.a == "OUTRO":
            d = d + 0
    r = s - d
    if self.a == "PREMIUM":
        z = r * 0.10
        d = d + z
        r = r - z
    f = 0.0
    if self.b == "RETIRADA":
        f = 0.0
    elif self.b == "NORMAL":
        if r >= 150.0:
            f = 0.0
        else:
            f = 12.0
    elif self.b == "EXPRESSA":
        f = 25.0
    t = r + f
```

```

print ( f"SUBTOTAL={s:.2 f}")
print ( f"DESCONTO={d:.2 f}")
print ( f"FRETE={f:.2 f}")
print ( f"TOTAL={t:.2 f}")

```

Esse método faz todos os cálculos e todas as verificações com relação ao pedido, fazendo a impressão dos resultados ao final da execução. Claramente, esse código é ilegível, não intuitivo e desorganizado.

2 Refatoração

Para iniciar a refatoração desse código, definiremos as classes de forma a criar maior abstração. Para isso adicionamos as classes *Item*, *Entrega*, *Cliente* e *Pedido*. Cada uma dessas classes terá seus próprios métodos e atributos.

Começando com a classe *Item*, ela "substituirá" a classe *I*, que contém três atributos. Esses atributos serão renomeados para *item*, *preco*, e *quantidade*. Agora, podemos adicionar um método *calculaDescontoItem()*, que calculará o desconto recebido para o item.

Veja abaixo a implementação da classe *Item*:

```

class Item:
    def __init__(self, item, preco, quantidade):
        self.item = item
        self.preco = preco
        self.quantidade = quantidade

    def calculaDescontoItem(self):
        if self.item == "LIVRO":
            return self.preco * self.quantidade * 0.05
        elif self.item == "ELETRONICO":
            return self.preco * self.quantidade * 0.02
        return 0.0

```

Agora, devemos implementar a classe *Cliente*, que será muito parecida com a classe *Item*. O cliente apenas terá um atributo chamado *tipo*, para determinar se o cliente é COMUM ou PREMIUM, e também será implementado um método *calculaDescontoCliente()* para calcular o desconto do cliente (10% para clientes PREMIUM).

Veja abaixo a implementação da classe *Cliente*:

```

class Cliente:
    def __init__(self, tipo=""):
        self.tipo = tipo

    def calculaDescontoCliente(self, subtotal):
        if (self.tipo == "PREMIUM"):
            return subtotal * 0.10
        return 0.0

```

Para a classe *Entrega*, também só teremos um único atributo chamado *tipo*, para definir o tipo da entrega, e o método *calculaFrete()* para calcular o valor do frete.

Veja abaixo a implementação da classe *Entrega*:

```
class Entrega:
    def __init__(self, tipo=""):
        self.tipo = tipo

    def calculaFrete(self, total):
        if ((self.tipo == "NORMAL") and (total < 150.0)):
            return 12.0
        elif (self.tipo == "EXPRESSA"):
            return 25.0
        return 0.0
```

Por fim, temos a classe *Pedido*, que é composta pelas outras três e faz os cálculos de preço da compra. A classe *Pedido* terá três atributos: *cliente* (objeto da classe *Cliente*), *entrega* (objeto da classe *Entrega*) e *carrinho* (lista de objetos da classe *Item*). Implementaremos também dois métodos: *calculaSubtotal()* e *checkout()*. O método *calculaSubtotal()* passará por todos os itens do carrinho, calcula o preço e desconto dos itens e retorna ambos. O método *checkout()* calcula todos os valores a serem impressos na saída do programa (subtotal, desconto, frete e total) e faz isso chamando os métodos implementados anteriormente. Depois, retorna esses quatro valores para serem impressos.

Veja abaixo a implementação da classe *Pedido*:

```
class Pedido:
    def __init__(self, cliente, entrega):
        self.cliente = cliente
        self.entrega = entrega
        self.carrinho = []

    def calculaSubtotal(self):
        subtotal = 0.0
        desconto = 0.0
        for item in self.carrinho:
            subtotal += item.preco * item.quantidade
            desconto += item.calculaDescontoItem()
        return subtotal, desconto

    def checkout(self):
        subtotal, descontoItens = self.calculaSubtotal()
        descontoCliente = self.cliente.calculaDescontoCliente(subtotal - descontoItens)
        desconto = descontoItens + descontoCliente
        frete = self.entrega.calculaFrete(subtotal - desconto)
        total = subtotal - desconto + frete

        return subtotal, desconto, frete, total
```

Após atualizar a *main()* para acomodar a refatoração das classes. Temos que o código continua com a mesma funcionalidade, mas está muito mais organizado, legível e abstraído. Os diferentes fatores de uma compra passaram a ser representados individualmente, com atributos e métodos representativos da sua classe, melhorando coesão e acoplamento. Também foi feita a separação das operações de entrada e saída das classes, com ambas acontecendo na *main()*.

Veja o diagrama UML abaixo, representando o programa refatorado:

